

agentic-ai

Project Concept Notes

Python foundations, object-oriented programming, data analysis, LangChain ingestion, embeddings, and vector search

Repository scope	A hands-on learning collection with notebooks, small scripts, sample data, and persisted FAISS artifacts.
Primary outcome	Build the foundations for document-based AI and retrieval-augmented workflows.
Prepared	12 August 2026

1. Project map

This repository is organized as a progression: learn Python, model data with classes and validation, analyze structured data, then ingest and search unstructured documents. It is not one end-to-end application; it is a set of focused experiments that show the pieces used in modern AI applications.

Core flow

Python	Data structures + functions + OOP	Data analysis	NumPy + Pandas
Document AI	Load -> split -> embed -> index -> retrieve	Validation	Pydantic models

Repository evidence

- python/ contains basics, OOP notebooks, file handling, exceptions, iterators, generators, decorators, and a small importable package.
- data_analysis/ contains NumPy and Pandas notebooks plus a product catalog CSV.
- langchain/ contains loaders for text, PDF, and web content; multiple text splitters; embedding experiments; and FAISS/Chroma vector-store examples.
- pydantic/ contains BaseModel examples for nested and constrained data models.

2. Python foundations

The basics section covers the language building blocks needed to write reliable data and AI workflows.

Key concepts

- Syntax and semantics: indentation, names, expressions, statements, and Python's dynamic typing.
- Data types: numbers, strings, booleans, None, lists, tuples, sets, and dictionaries.
- Control flow: if/elif/else, for loops, while loops, break, continue, and comprehensions.
- Functions: positional and keyword arguments, defaults, *args, **kwargs, return values, and scope.
- Functional tools: lambda expressions for short functions, map() for transformation, and filter() for selection.
- Modules and files: import statements, package structure, pathlib/os paths, text/binary I/O, JSON, CSV, and datetime utilities.
- Exceptions: try/except/else/finally, raising errors, and defining domain-specific exception classes.

Example: transform then filter

```
numbers = [1, 2, 3, 4, 5, 6]
doubled = list(map(lambda n: n * 2, numbers))
even = list(filter(lambda n: n % 2 == 0, doubled))
# [2, 4, 6, 8, 10, 12]
```

Practical takeaway: keep transformations small and composable. For more complex logic, prefer named functions so the intent is easy to test and debug.

3. Object-oriented programming

The OOP notebooks use familiar examples such as cars, animals, shapes, people, and bank accounts to show how behavior and state are grouped.

Concepts represented

- Classes and objects: a class defines structure/behavior; an object is a concrete instance.
- `__init__`: initializes instance state. `self` refers to the current object.
- Encapsulation: protect or control access to state through methods and properties.
- Abstraction: expose a useful interface while hiding implementation details; abstract base classes define required methods.
- Inheritance: derive a specialized class from a base class and reuse or extend behavior.
- Polymorphism: different classes can respond to the same method call in their own way.
- Magic methods: `__str__` and `__repr__` control representation; `__add__`, `__sub__`, and `__eq__` customize operators.
- Iterators and generators: produce values lazily with `__iter__`/`__next__` or `yield`, reducing memory use.
- Decorators: wrap a function to add behavior without changing its core implementation.

Example: a common interface

```
class Shape:
    def area(self):
        raise NotImplementedError

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width, self.height = width, height

    def area(self):
        return self.width * self.height
```

The caller can use `shape.area()` without needing to know whether the object is a rectangle, circle, or another shape. That same interface-based thinking is useful for loaders, embedders, and retrievers.

4. NumPy and Pandas

The data analysis section introduces array-oriented numerical work and tabular data manipulation.

NumPy

- `ndarray`: a homogeneous, multidimensional array optimized for numerical operations.
- Vectorization: operate on whole arrays instead of writing Python loops for every element.
- Shape and dimensions: understand the size and layout of arrays before combining them.
- Typical uses: arithmetic, reshaping, slicing, aggregation, and preparing numerical inputs.

Pandas

- Series: a labeled one-dimensional sequence. DataFrame: a labeled two-dimensional table.
- Loading data: read CSV files into DataFrames and inspect columns, types, and missing values.
- Selection: select columns, rows, and filtered subsets using labels or boolean conditions.
- Transformation: create columns, apply functions, sort, group, aggregate, and clean values.
- Analysis pattern: inspect -> clean -> transform -> summarize -> export or visualize.

Example: tabular filtering

```
import pandas as pd

df = pd.read_csv("products_catalog.csv")
selected = df[df["price"] > 100]
summary = selected.groupby("category")["price"].mean()
```

5. Pydantic data modeling

Pydantic is used to define typed models and validate input data at boundaries. This is especially important when data comes from files, APIs, users, or language-model output.

Key ideas

- BaseModel: declare a schema as a Python class with typed fields.
- Parsing and validation: incoming values are checked and, where appropriate, converted to the declared types.
- Optional fields: represent values that may be absent.
- Nested models: compose Address, Customer, Employee, or classroom-style structures from smaller models.
- Field metadata and constraints: add descriptions, defaults, and validation rules.
- Serialization: convert validated models to dictionaries or JSON for downstream tools.

Example

```
from pydantic import BaseModel, Field

class Item(BaseModel):
    name: str
    price: float = Field(gt=0)

item = Item(name="Notebook", price=12.5)
```

Design principle: validate at the edge, then let the rest of the pipeline work with trusted, structured values.

6. LangChain document ingestion

The LangChain notebooks demonstrate the front half of a retrieval-augmented generation workflow: turn different source formats into a common Document representation.

Loaders used

- TextLoader: reads plain text such as speech.txt.
- PyPDFLoader: reads PDF pages and attaches metadata such as source and page number.
- WebBaseLoader: fetches web content and can use BeautifulSoup filters to keep relevant HTML sections.
- Document: combines page_content with metadata, allowing later stages to preserve provenance.

Conceptual pipeline

```
source files / web pages
|
v
Document(page_content, metadata)
|
v
text chunks with source/page metadata
```

Metadata matters because retrieval results should be traceable. A useful answer can point back to the source document or page that supplied the context.

7. Chunking and embeddings

Long documents are split before embedding because models and vector indexes work better with focused, bounded pieces of text.

Text splitters

- `CharacterTextSplitter`: split using a chosen separator and size/overlap settings.
- `RecursiveCharacterTextSplitter`: tries a hierarchy of separators so chunks preserve natural boundaries when possible.
- `HTMLHeaderTextSplitter`: uses HTML headings to keep section structure in metadata.
- `RecursiveJsonSplitter`: breaks nested JSON while respecting its structure.
- `chunk_size` controls approximate chunk length; `chunk_overlap` repeats a small boundary region so context is not lost between chunks.

Embeddings

- An embedding is a numeric vector representing semantic meaning.
- The notebooks compare Google Gemini, Hugging Face, Ollama, and other embedding integrations.
- Similar meanings produce vectors that are close under a similarity metric, enabling semantic rather than keyword-only search.

Example

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
)
chunks = splitter.split_documents(docs)
# embed chunks before indexing
```

8. Vector stores and retrieval

The FAISS and Chroma examples show how embedded chunks become searchable knowledge. FAISS is also persisted in the repository as `faiss_index/index.faiss` and `index.pkl`.

Retrieval flow

- Indexing: compute an embedding for each chunk and store the vectors with the original document data.
- Similarity search: embed a user query, compare it with indexed vectors, and return the nearest chunks.
- Persistence: save the index locally and load it again for later sessions.
- RAG pattern: retrieve relevant chunks, place them in a prompt, and ask a language model to answer from that context.
- Trade-off: smaller chunks improve precision but may lose context; larger chunks preserve context but can dilute relevance.

End-to-end mental model

```
documents -> loaders -> chunks -> embeddings -> vector store
                                     ^
user question -> query embedding -> similarity search -> context -> answer
```

The project focuses on ingestion, representation, and retrieval. A production RAG system would add prompt construction, an LLM call, citations, evaluation, access control, and monitoring.

9. Dependencies and study checklist

The requirements file names the main ecosystem components: NumPy, Pandas, Pydantic, LangChain, python-dotenv, PDF/HTML loaders, Chroma, FAISS, Ollama, Hugging Face, and Google Gemini integrations.

Suggested learning order

- 1. Practice functions, collections, exceptions, file paths, and modules.
- 2. Build small classes, then add inheritance, polymorphism, and validation.
- 3. Load and inspect a CSV with Pandas; use NumPy for numerical transformations.
- 4. Load a text file and a PDF into LangChain Documents.
- 5. Split documents with different strategies and compare chunk quality.
- 6. Generate embeddings with one local or hosted provider.
- 7. Create a FAISS or Chroma index, persist it, and test semantic retrieval.
- 8. Add a prompt and language-model step only after retrieval quality is understandable.

Common pitfalls

- Use environment variables for API keys and never commit secrets.
- Pin compatible package versions for reproducibility.
- Keep source metadata with every chunk so retrieved context can be explained.
- Treat `allow_dangerous_deserialization` as a trust decision when loading persisted indexes.
- Inspect encoding issues in source documents; the repository contains examples with mixed character encodings.
- Measure retrieval with representative questions instead of assuming a working index is a useful index.

Summary

The project teaches the full set of building blocks behind a simple document-AI system: Python provides the language and abstractions, Pandas/NumPy handle structured data, Pydantic makes boundaries reliable, and LangChain connects documents to chunking, embeddings, and vector retrieval. Together they form a practical foundation for an agentic or retrieval-augmented application.